topazus

解説書

証明すると速くなる,Lisp 記法の競プロ言語
—— 仕様の全体像と,その背景にある知識 ——

grammer.md(2026 年 11 月 11 日時点)に基づく

目次

1 この文書について 3
2 topazus の全体像 4
3 コンパイラの仕組み 5
4 Lisp と s 式 7
5 関数型プログラミングの基礎 8
6 型の基礎 9
7 数値の型 10
8 篩型(条件付きの型) 11
9 依存型 12
10 カリー=ハワード同型対応 13
11 証明の書き方 14
12 停止性 15
13 証明の消去と数量 16
14 線形型とミュータブル配列 17
15 メモリ管理 18
16 メモリ安全性 19
17 末尾呼び出しとスタック 20
18 コード生成 21
19 他の言語との位置づけ 22
20 未決定事項 23
21 次の一歩 24
22 参考資料 25
23 用語集 26
24 コンパイラを作るときに考えること 27

1 この文書について

topazus は「言語を作ってみたい」という気持ちから始まった言語である.設計を進めるうちに,依存型・線形型・篩型・参照カウントといった,プログラミング言語の研究で使われる言葉がたくさん出てきた.この文書は,それらを1つずつ説明し,grammer.md に書いてある仕様が「なぜそうなっているのか」を理解できるようにするためのものである.

各章は次の形で書いてある.

- **topazus では** … grammer.md での書き方
- **背景知識** … その仕組みが何で,なぜ必要か
- **他の言語では** … 似た仕組みを持つ言語との比較

全部を一度に理解する必要はない.第 2 章(全体像)と第 3 章(コンパイラの仕組み)を読んだら,あとは気になった章から読めばよい.

2 topazus の全体像

topazus を一言で言うと,「**証明すると速くなる,Lisp 記法の競プロ言語**」である.これは 3 つの柱からできている.

柱	内容	関係する章
書きやすさ	Lisp 風の括弧の記法.イギリス英語と皮肉なエラー文で個性を出す	4, 5
証明	Lean4 や Agda のように数学的な性質を証明できる	8〜13
速度	証明を使って実行時チェックを消す.GC を使わない	7, 14〜18

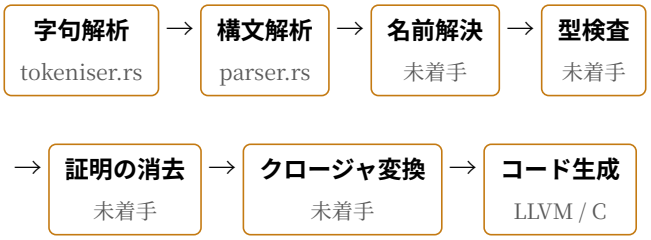
この 3 つは別々のものではなく,つながっている.たとえば配列の添字が範囲内であることを**証明**できれば,実行時の範囲チェックを省けるので**速く**なる.これが topazus の一番の特徴である.

> **ポイント**
> 多くの言語では「安全」と「速さ」はトレードオフになる.Python は安全だが遅く,C は速いが危ない.Rust はその間を狙った言語である.topazus は「証明」という道具を使って,安全さを保ったまま実行時のコストを削ることを目指す.

3 コンパイラの仕組み

3.1 コンパイラは何段階かに分かれている

コンパイラは,ソースコードを一度に機械語へ変換するのではなく,いくつかの段階(パス)に分けて少しずつ変換する.topazus では次のような流れになる.



1. **字句解析(lexer, tokeniser)**:文字の並びを「単語」(トークン)に区切る.`(let [i64 n] <- 10)` は `(`, `let`, `[`, `i64`, `n`, `]`, `<-`, `10`, `)` になる.
2. **構文解析(parser)**:トークンの並びを木構造(AST,抽象構文木)にする.「これは let 文で,型は i64,名前は n,値は 10」という形にまとめる.
3. **名前解決**:変数名が,どこで定義されたものを指しているかを結びつける.同じ名前が別の場所で使われていても区別できるようにする.
4. **型検査**:型が合っているかを調べる.topazus ではここで篩型の条件や証明も検査する.一番大きな部分になる.
5. **証明の消去**:型検査が終わったら,証明は不要なので取り除く(第 13 章).
6. **クロージャ変換**:関数の中で定義された関数を,外側の関数から独立した形に直す(第 18 章).
7. **コード生成**:LLVM IR や C 言語のコードを出力する.

3.2 今どこまでできているか

`src/` を見ると,字句解析(tokeniser.rs)はひととおり動き,構文解析(parser.rs)は書いている途中である(現在はコンパイルエラーが出る状態).

grammer.md の仕様が増えたことで,字句解析にも直すところが出てきた.

問題	内容
識別子に数字が使えない	`i64` が `i` と `64` に分かれてしまう.`add-comm`,`get?`,`set!` も同様.識別子の 2 文字目以降に数字・`-`・`?`・`!` を許す必要がある
`==` が無い	`=` 1 文字しか認識しないので,`==` は `=` が 2 つになる
`<-` と負の数	`(< x -1)` が `<-` と `1` に分かれてしまう.「`<-` の前後には空白が必要」などのルールを決める
文字列・文字	`"` や `'` を 1 つのトークンにしているだけで,中身をまとめて読んでいない
知らない文字	`∀` などの知らない文字が黙って無視される.エラーにすべき(皮肉を込めて)
新しい記号	`+%` などの折り返し演算子,篩型の `|` を追加する

おすすめ

最初から LLVM まで作ろうとすると,動くものが見えないまま時間がかかる.まずは **AST をそのまま実行するインタプリタ**を作り,FizzBuzz が動くところまで持っていくのがおすすめである.型検査や LLVM は,動くものができてから足していけばよい.詳しくは第 21 章.

4 Lisp と s 式

4.1 topazus では

関数呼び出しも,変数宣言も,すべて括弧で書く.

```
(println "Hello, World!")
(+ 1 2)
(let [i64 n] <- 10)
```

4.2 背景知識:前置記法

`(+ 1 2)` のように,演算子を先頭に書く書き方を**前置記法**(ポーランド記法)という.普通の数式 `1 + 2 * 3` は**中置記法**で,`*` が `+` より先に計算されるという「優先順位」のルールが必要になる.前置記法では `(+ 1 (* 2 3))` と括弧で構造がそのまま書かれるので,優先順位を考える必要がない.

括弧で囲まれた式を **s 式**(S-expression)という.Lisp(1958 年)から続く書き方である.

4.3 なぜ Lisp 風の記法は作りやすいか

s 式は「括弧の中の最初の要素を見れば,何の式か分かる」.`let` なら変数宣言,`if` なら分岐,それ以外なら関数呼び出し,と分岐するだけでよい.そのため構文解析がとても簡単になる.parser.rs の `parse_sexpr` がまさにこれをやっている.

4.4 発展:同図像性とマクロ

Lisp では,プログラム自体がリスト(データ)と同じ形をしている.これを**同図像性**(homoiconicity)という.この性質を使うと,「プログラムを受け取ってプログラムを返す関数」=**マクロ**を簡単に作れる.たとえば `unless` という構文を,言語に組み込まなくてもユーザーが自分で定義できる.

topazus では型の角括弧 `[i64 n]` などがあるので純粋な Lisp ほど単純ではないが,将来マクロを入れると,Rust など他の言語との大きな違いになる.

5 関数型プログラミングの基礎

5.1 代入ではなく束縛

grammer.md には「変数に値を“代入”するではなく“束縛”すると考えてほしい」とある.

- **代入**:箱(メモリ)があって,そこに値を入れる.あとで別の値に入れ替えられる.
- **束縛**:値に名前を付ける.一度付けた名前は,同じ値を指し続ける.

数学で「$x = 3$ とおく」と書いたら,その議論の中で x はずっと 3 である.束縛はこれと同じ考え方である.値が途中で変わらないので,プログラムを読むときに「この時点で x はいくつか」を追いかける必要がない.これが**イミュータブル**(不変)の利点である.

5.2 関数も値

topazus では,関数の定義は「無名関数を変数に束縛すること」として説明されている.

```
(let [fn sqr] <- [i64 r] [i64 x] -> | (* x x) |)
```

右側の `[i64 r] [i64 x] -> | (* x x) |` が無名関数(名前のない関数),それに `sqr` という名前を付けている.関数を数値や文字列と同じように変数に入れたり,引数として渡したりできることを,**関数が第一級の値である**という.

5.3 関数合成

数学では,2 つの関数 f, g から $(f \circ g)(x) = f(g(x))$ という新しい関数を作れる.これが**関数合成**である.topazus の `synth` はこれをそのまま表している.

```
(synth [fn print_sqr] <- print, sqr)   // print ∘ sqr
```

$f \circ g$ が「g を先に,f を後に」適用するのと同じく,`synth` も右に書いた関数から先に実行される.

5.4 クロージャ

関数の中で,外側の変数を使う関数を作ることができる.

```
(let [fn make_adder] <- [fn r] [i64 n] -> | ([i64 r] [i64 x] -> | (+ x n) |) |)
```

内側の無名関数は,外側の `n` を覚えている.このように「関数」と「その関数が使う外側の変数」をセットにしたものを**クロージャ**という.これは後でコード生成の際に問題になる(第 18 章).

6 型の基礎

6.1 型とは

型は「その値がどんな種類のデータか」を表す.`10` は整数,`"abc"` は文字列.型があると,「整数と文字列を足す」のような間違いを実行前に見つけられる.

- **静的型付け**:実行前(コンパイル時)に型を調べる.Rust,Haskell,Lean,topazus.
- **動的型付け**:実行中に型を調べる.Python,JavaScript,多くの Lisp.

6.2 型推論

topazus は今のところ型を全部書く方式だが,将来「書かなくてもコンパイラが型を推測する」機能(**型推論**)を入れることもできる.

loadmap.md にある **Hindley-Milner(HM)型推論**は,Haskell や OCaml で使われている方式で,型注釈を 1 つも書かなくても全部の型を推論できる.ただし**依存型(第 9 章)には使えない**.依存型があると,型の中に値や計算が入ってくるので,HM の前提が崩れるからである.

依存型を持つ言語(Lean,Agda,Idris)では,**双方向型検査**(bidirectional type checking)という方式を使う.これは次の 2 つのモードを組み合わせる.

- **検査モード**:「この式はこの型のはず」と型が分かっている状態で,式を調べる.
- **推論モード**:型が分からない状態で,式から型を求める.

関数の引数の型など「人間が書いた型」を手がかりにして,そこから内側へ型を伝えていくイメージである.topazus は型を全部書く方式なので,双方向型検査と相性がよい.

> **設計上の注意**
> 最終的に依存型まで作るなら,最初から HM ではなく双方向型検査で作っておくと,あとで型検査を作り直す手間が減る.

7 数値の型

7.1 topazus では

数値の型は「数学用」と「計算用」に分かれている.

```
数学用: [nat] [int]                  // 上限なし
計算用: [i8] [i16] [i32] [i64]       // 符号付き
        [u8] [u16] [u32] [u64]        // 符号なし
        [f32] [f64]                   // 浮動小数点
```

7.2 背景知識:コンピュータの整数には上限がある

CPU は決まったビット数で整数を扱う.n ビットで表せる整数の範囲は次のとおり.

型	最小値	最大値
`u32`	0	$2^{32}-1 \approx 4.3 \times 10^9$
`i32`	$-2^{31} \approx -2.1 \times 10^9$	$2^{31}-1 \approx 2.1 \times 10^9$
`i64`	$-2^{63} \approx -9.2 \times 10^{18}$	$2^{63}-1 \approx 9.2 \times 10^{18}$

符号付き整数は **2 の補数**という形式で表される.この形式では,最大値に 1 を足すと最小値に戻ってしまう.たとえば `i32` で $2147483647+1$ を計算すると -2147483648 になる.これを**オーバーフロー**という.

> **競プロでよくある失敗**
> 10^5 個の 10^5 以下の数の積や和を `int`(32 ビット)で計算して,オーバーフローで答えが合わない(WA).C++で `long long` を使えと言われるのはこのためである.

7.3 なぜ 2 種類に分けるのか

- **数学用の型**(`nat`, `int`)は上限がない.証明では「$a+b=b+a$」のような性質を扱うが,上限があるとオーバーフローの場合を毎回考えなければならず,証明が非常に面倒になる.
- **計算用の型**(`i64` など)は CPU がそのまま計算できるので速い.

topazus では,証明は数学用の型で書き,実行は計算用の型で行う.数学用から計算用に変換するときは「範囲に収まる」ことを証明する必要がある.

7.4 他の言語では

- **Python, Haskell の `Integer`, Lean の `Nat`**:上限なし(多倍長整数).内部で必要に応じて桁を増やすので,計算用の整数より遅い.
- **Rust**:デバッグビルドではオーバーフローでパニックする.リリースビルドでは黙って折り返す.
- **C/C++**:符号付き整数のオーバーフローは**未定義動作**(第 16 章)で,何が起きてもおかしくない.

8 篩型(条件付きの型)

8.1 topazus では

型の後に | で条件を書く.

```
(let [i64 n | (> n 0)] <- 10)   // 正の整数
(let [fn abs] <- [i64 r | (>= r 0)] [i64 x] -> | ... |)
```

8.2 背景知識

ある型の値のうち,条件を満たすものだけを集めた型を**篩型**(ふるいがた,refinement type)という.数学の集合の書き方 $\{n \in \mathbb{Z} \mid n > 0\}$ とそっくりである.

コンパイラは,プログラムのあちこちで条件が満たされているかを確認する.たとえば `abs` の場合,

- `x < 0` のとき返り値は `-x`.`x < 0` なら `-x > 0` なので,`r >= 0` を満たす.
- それ以外のとき返り値は `x`.`x >= 0` なので,`r >= 0` を満たす.

という推論を自動で行う.

8.3 どうやって自動で確認するのか

条件を論理式に直して,**SMT ソルバー**という「論理式が成り立つかを自動で判定するプログラム」に渡すのが一般的である.代表的なものに Microsoft の **Z3** がある.

最初から SMT ソルバーを使う必要はない.`(< i n)` のような単純な不等式だけなら,自分で検査を書くこともできる.まずは簡単な範囲で作り,足りなくなったら Z3 を使う,という進め方がよい.

8.4 返り値に名前がある理由

topazus の関数は返り値にも名前(`r` など)を付ける.これは返り値の条件 `(>= r 0)` を書くためである.最初は何となく付けていた名前が,篩型によって意味を持つようになった.

8.5 他の言語では

- **Liquid Haskell**:Haskell に篩型を追加したもの.SMT ソルバーを使う.
- **F***(エフスター):Microsoft の研究用言語.篩型と依存型を両方持つ.

9 依存型

9.1 topazus では

配列の型に長さが入っている.

```
[array i64 5 a]                    // 長さ5の配列
[array i64 n a, i64 i | (< i n)]   // 長さnの配列と,n未満の添字
```

9.2 背景知識

普通の型は「整数」「文字列」のように,値の種類だけを表す.**依存型**(dependent type)は,型の中に**値**が入る型である.`[array i64 n]` の `n` は型ではなく値(数)であり,「長さが `n` の配列」という型になっている.型が値に「依存」しているので依存型という.

依存型を使うと,次のようなことを型で表せる.

- 「長さ n の配列と長さ m の配列をつなげると,長さ $n+m$ の配列になる」
- 「ソート関数の返り値は,ソート済みの配列である」

9.3 依存型の難しさ

型の中に値や計算が入るので,型検査の途中で**計算**が必要になる.たとえば `[array i64 (+ 2 3)]` と `[array i64 5]` が同じ型かを判断するには,`(+ 2 3)` を計算して `5` にする必要がある.そのため依存型の型検査器は,小さなインタプリタを中に持つことになる.

10 カリー=ハワード同型対応

依存型を持つ言語でなぜ「証明」ができるのか.その答えが**カリー=ハワード同型対応**(Curry-Howard correspondence)である.

10.1 命題は型,証明はプログラム

この対応は,「論理学の命題」と「プログラムの型」が同じ構造を持っている,という発見である.

論理学	プログラム	意味
命題 P	型 `P`	
P の証明	型 `P` を持つ値	値が作れる = 証明できた
$P \to Q$(ならば)	関数 `P -> Q`	P の証明から Q の証明を作る
$P \wedge Q$(かつ)	タプル `(P, Q)`	両方の証明を持つ
$P \vee Q$(または)	enum(どちらか)	どちらかの証明を持つ
$\forall x.P(x)$(すべての)	依存型の関数	どんな x を渡しても $P(x)$ の証明を返す
偽(矛盾)	値が 1 つもない型	証明が作れない

つまり「定理を証明する」ことは「その型を持つ値(プログラム)を書く」ことと同じになる.型検査器が「この値はこの型を持つ」と確認することが,そのまま「この証明は正しい」と確認することになる.

10.2 topazus の定理

```
(theorem add-comm [∀ nat a, nat b] (== (+ a b) (+ b a))
  | (induction a ...) |)
```

これは「すべての自然数 a, b について $a + b = b + a$」という命題(型)と,その証明(値)である.

11 証明の書き方

11.1 数学的帰納法

自然数についての性質を証明するときの基本が**数学的帰納法**である.

1. $P(0)$ が成り立つことを示す.
2. $P(k)$ が成り立つと仮定して,$P(k+1)$ が成り立つことを示す.

この 2 つを示せば,すべての自然数 n で $P(n)$ が成り立つ.

プログラムで言うと,これは**再帰関数**と同じ形をしている.「0 の場合の答え」と「k の答えを使って $k+1$ の答えを作る方法」を書く.カリー=ハワード同型対応で見ると,帰納法による証明は再帰関数そのものである.

11.2 証明項とタクティク

証明の書き方には 2 つのスタイルがある.

- **証明項**:証明をプログラム(値)として直接書く.Agda はこちらが中心.
- **タクティク**:「帰納法を使う」「式を簡単にする」のような命令を並べて,コンパイラに証明を組み立ててもらう.Lean,Rocq(旧 Coq)はこちらが中心.

```
-- Lean 4 のタクティクの例
theorem add_comm (a b : Nat) : a + b = b + a := by
  induction a with
  | zero => simp
  | succ k ih => omega
```

topazus の `(induction a ...)` や `(by add-comm)` はタクティク風の書き方である.どちらのスタイルにするかは,まだ決まっていない(第 20 章).

> **作りやすさ**
> 実装としては証明項のほうが簡単である.型検査器だけあれば証明を検査できる.タクティクは「証明を組み立てる仕組み」を別に作る必要がある.まず証明項で作り,あとからよく使うタクティクを足していく方法もある.

12 停止性

12.1 topazus では

証明に使う関数は必ず停止しなければならない.停止性を検査しない関数には `partial` を付ける.

```
(let partial [fn solve] <- [i64 r] [i64 n] -> | ... |)
```

12.2 なぜ停止性が必要なのか

もし無限ループする関数を証明に使えると,何でも「証明」できてしまう.

```
(let [fn bad] <- [P r] [void] -> | (bad) |)
```

この関数は「型 `P` の値を返す」と言っているが,実際には自分自身を呼び続けて永遠に終わらない.しかし型だけ見ると「`P` の値を返す関数」なので,`P` がどんな命題でも(たとえ $1 = 2$ でも)「証明できた」ことになってしまう.これでは証明の意味がなくなる.

12.3 どうやって停止性を検査するか

一般に「任意のプログラムが停止するか」を判定することは**不可能**である(停止性問題,チューリング 1936 年).そこで,安全側に倒した簡単なルールを使う.

代表的なのが**構造的再帰**のルールである.「再帰呼び出しの引数が,元の引数より構造的に小さくなっている」ことを確認する.たとえば n についての関数が $n - 1$ で自分を呼ぶなら,いつか 0 にたどり着くので停止する.

このルールで確認できない関数(本当は止まるが,コンパイラには分からない関数も含む)は `partial` を付けて書く.

12.4 競プロとの関係

競プロのコードは while ループや複雑な再帰が多く,停止性を証明するのは大変である.そのため topazus では「普通のコードは `partial` で書いてよい」としている.`partial` は停止性を検査しないだけで,型やメモリ安全性は保たれる.

13 証明の消去と数量

13.1 証明は実行時には要らない

証明は「コンパイル時に正しさを確認するため」のものなので,確認が終われば プログラムの実行には不要である.そこでコンパイラは,型検査の後に証明をすべて取り除く.これを**消去**(erasure)という.証明をいくら書いても実行速度は変わらない.

13.2 topazus の数量

型の前に数量を書くことで,その値を何回使うかを指定する.

数量	意味
`0`	証明専用.コンパイル時だけ使い,実行時には消える
`1`	線形.ちょうど 1 回だけ使う(第 14 章)
`ω`	制限なし(省略時)

13.3 背景知識:数量型理論

この仕組みは**数量型理論**(Quantitative Type Theory, QTT)と呼ばれ,**Idris 2** で使われている.1 つの仕組みで「消してよいもの(0)」と「書き換えてよいもの(1)」の両方を表せるのが特徴である.

14 線形型とミュータブル配列

14.1 topazus では

数量 1 の配列は,その場で書き換えられる.

```
(let [fn init] <- [1 array i64 n r] [1 array i64 n a] -> | (set! a 0 1) |)
```

14.2 なぜ書き換えが必要なのか

関数型言語では値を変更しないのが基本である.配列の 1 か所を変えたいときも,配列全体をコピーして新しい配列を作る.しかし長さ 10^5 の配列で DP をするたびにコピーしていたら,計算量が $O(n)$ 倍になり,競プロでは間に合わない.

14.3 線形型

線形型は「この値はちょうど 1 回だけ使う」ことを型で保証する仕組みである.

値を書き換えて困るのは,その値を他の誰かも見ている場合である.線形型の値は 1 回しか使えないので,他の誰かが見ていることはありえない.だから,コピーせずにその場で書き換えても,外から見て「新しい値を作った」のと区別がつかない.

`set!` は配列 `a` を使い切って,書き換えた配列を返す.`a` はもう使えないので,書き換え前の `a` を誰かが見ることはない.

14.4 Rust との関係

Rust の**所有権**も似た考え方である.正確には,Rust は「たかだか 1 回使う」(使わずに捨ててもよい)という**アフィン型**である.線形型は「ちょうど 1 回」なので,少し厳しい.

15 メモリ管理

15.1 スタックとヒープ

プログラムが使うメモリには 2 種類ある.

- **スタック**:関数の引数やローカル変数を置く場所.関数が終わると自動で片付く.速いが小さい.
- **ヒープ**:配列やリストなど,大きさが実行時に決まるデータを置く場所.いつ片付けるかを誰かが決める必要がある.

ヒープのデータをいつ片付けるかが**メモリ管理**の問題である.

15.2 3 つの方式

方式	仕組み	欠点	使う言語
GC(ガベージコレクション)	定期的に使われていないデータを探して片付ける	片付ける間プログラムが止まることがある	Java, Haskell, OCaml
参照カウント	データごとに「何か所から参照されているか」を数え,0 になったら片付ける	数を増減するコスト.循環参照を片付けられない	Swift, Python, Lean 4
所有権・借用	コンパイル時に片付けるタイミングを決める	書くのが難しい(借用チェッカーとの戦い)	Rust

15.3 topazus の方式

topazus は**参照カウント**を使い,さらに 2 つの工夫をする.

1. 参照カウントが 1(自分しか見ていない)なら,コピーせずにその場で書き換える.
2. 数量 1 の値は参照カウントが 1 だと型で分かっているので,数える処理自体を省く.

この方法は Lean 4(論文 Counting Immutable Beans)や Koka(論文 Perceus)で使われている.

> **参照カウントの弱点が問題にならない理由**
> 参照カウントは,A が B を指し,B が A を指す「循環参照」を片付けられない.しかしイミュータブルなデータでは,作った後に参照を書き換えられないので,循環参照がほとんどできない.関数型言語と参照カウントは相性がよい.

16 メモリ安全性

16.1 未定義動作

C言語では,配列の範囲外を読んだり,解放したメモリを使ったりすると**未定義動作**になる.未定義動作とは「何が起きても言語仕様は責任を持たない」という意味で,クラッシュする,変な値が出る,何も起きないように見える,のどれもありうる.セキュリティ上の脆弱性の大きな原因でもある.

メモリ安全な言語とは,このような未定義動作が起きないことを保証する言語である.

16.2 定理証明系の言語はメモリ安全か

Lean,Rocq,Agda はもともとメモリ安全である.GC や参照カウントでメモリを管理し,生のポインタを持たない.そのため「メモリ安全」だけでは差別化にならない.Python,Java,Haskell もメモリ安全である.

16.3 topazus が目指すもの

言語	範囲外アクセス	オーバーフロー	メモリ解放
Lean / Haskell	実行時チェック or 証明	問題なし(多倍長)	GC・参照カウント
Rust	実行時チェック(パニック)	リリースでは折り返す	借用チェッカー
topazus	証明でチェック不要	証明で起きないと保証	参照カウント+線形型

Rust は「未定義動作が起きない」ことは保証するが,パニック(実行時エラーで停止)は起こりうる.topazusは,パニックが起きないこと自体を証明で保証し,しかも実行時チェックを省くことを目指す.競プロで言えば「RE で落ちない」「オーバーフローで WA にならない」ことを,提出前にコンパイラが保証する.

16.4 証明できないときは

毎回証明を書くのは大変なので,証明できない場合の逃げ道も用意してある.

- 添字:`get?` を使い,失敗の可能性を型で表す.
- オーバーフロー:実行時チェックが入り,オーバーフローしたら停止する.黙って折り返すことはない.
- 意図的に折り返したいとき(ハッシュなど):`+%` などの折り返し演算子を使う.
- 外部関数など保証できないもの:`(unsafe | ... |)` で囲む.

17 末尾呼び出しとスタック

17.1 スタックオーバーフロー

関数を呼ぶたびに,スタックに引数や戻り先が積まれる.再帰が深くなるとスタックが溢れてプログラムが落ちる.これが**スタックオーバーフロー**である.競プロでは,10^5 頂点のグラフを再帰 DFS で探索するときによく起きる.

17.2 末尾呼び出しの最適化

関数の**最後の処理**が別の関数の呼び出しであるとき,これを**末尾呼び出し**という.

```
// 末尾呼び出し: 最後にやるのが sum 自身の呼び出し
(let partial [fn sum] <- [i64 r] [i64 n, i64 acc] ->
  | (if (== n 0) { then -> acc, else -> (sum (- n 1) (+ acc n)) }) |)
```

末尾呼び出しの場合,呼び出した後に元の関数でやることが残っていないので,スタックを積まずに済む.つまり再帰をループに変換できる.関数型言語ではループの代わりに再帰をよく使うので,この最適化はほぼ必須である.

18 コード生成

18.1 LLVM

LLVM は,多くのコンパイラが使っている「コンパイラの後半部分」である.Rust,Swift,Clang(C/C++) などが LLVM を使っている.LLVM IR(中間表現)という,アセンブリに近い言語を出力すれば,あとは LLVM が最適化して各 CPU 向けの機械語にしてくれる.

```
; (+ a b) を返す関数を LLVM IR で書いたもの
define i64 @add(i64 %a, i64 %b) {
  %r = add i64 %a, %b
  ret i64 %r
}
```

18.2 C への変換という選択肢

LLVM の代わりに **C 言語のコード**を出力する方法もある.出力した C を gcc や clang でコンパイルすれば,LLVM を直接使うのと同じくらい速くなる.

競プロでは特に重要である.AtCoder などのジャッジに自作言語は入っていないので,自作言語で提出するには「C に変換したコードを提出する」のが現実的な方法になる.

18.3 クロージャ変換

LLVM や C の関数は,外側の変数を覚えられない.そこでクロージャ(第 5 章)を「関数ポインタ」と「使う外側の変数をまとめた構造体(環境)」のペアに変換する.これを**クロージャ変換**という.

```
// make_adder が返すクロージャは,C ではこんな形になる
struct env { int64_t n; };
int64_t adder_body(struct env* e, int64_t x) { return x + e->n; }
```

19 他の言語との位置づけ

言語	topazus との関係
Rust	メモリ安全で速い.所有権(アフィン型).証明はできない
Haskell / OCaml	関数型言語の代表.HM 型推論.GC を使う
Lean 4	定理証明支援系で,普通のプログラムも書ける.参照カウントとその場書き換え.AtCoder でも使える
Rocq(旧 Coq)	定理証明支援系の老舗.タクティクで証明する
Agda	依存型の研究用言語.証明項で書く
Idris 2	数量型理論(0・1・ω).topazus の数量はここから
ATS	証明と速度の両立を狙った言語で,目標は topazus に最も近い.構文が難しいことで有名
Liquid Haskell / F*	篩型と SMT ソルバー
Koka	参照カウント+その場書き換え(Perceus)

topazus の立ち位置は「**ATS の目標を,Lisp の書きやすい記法と競プロ向けの実用性で実現する**」とまとめられる.

20 未決定事項

grammer.md を書く中で,まだ決まっていないことを挙げておく.

- **struct / enum**:ユーザー定義の型.enum はパターンマッチと組み合わせて使う.match ですべての場合を書いたか(網羅性)の検査もほしい.
- **証明のスタイル**:証明項かタクティクか(第 11 章).
- **`let mut` と線形型の関係**:両方とも「書き換え」に関わる.`let mut` を残すか,線形型に一本化するか.
- **配列の型の書き方**:`[array i64 5 a]` は「最後の要素が名前」というルールで読むが,長さに式を書くと読みにくい.
- **`|` の使い分け**:篩型の条件と関数本体の両方で `|` を使っている.構文解析では文脈で区別できるが,人間にとって紛らわしくないか.
- **スタックサイズの指定方法**.
- **標準ライブラリ**:競プロで使うデータ構造(優先度付きキュー,ハッシュマップなど)をどう用意するか.

21 次の一歩

おすすめの順番は次のとおりである.各段階で「動くもの」ができるようにしてある.

1. **字句解析を直す**(第 3 章の表).
2. **構文解析を完成させる**.まずは `let`,関数,`if`,`match`,`loop` だけでよい.型の条件や証明は後回し.
3. **ツリーウォーキング・インタプリタを作る**.AST を直接たどって実行する.FizzBuzz が動けば大きな一歩.
4. **単純な型検査を作る**.篩型や依存型なしの,普通の型だけ.ただし双方向型検査の形で作る.
5. **C へのコード生成**.インタプリタと同じ結果が出るかで正しさを確認できる.
6. **篩型**:`(< i n)` などの単純な不等式から.
7. **依存型と証明**:ここで型検査を拡張する.
8. **LLVM**:必要になったら.

22 参考資料

22.1 コンパイラ全般

- 『低レイヤを知りたい人のための C コンパイラ作成入門』(植山類):日本語.手書きコンパイラの作り方を一歩ずつ説明している.https://www.sigbus.info/compilerbook
- Crafting Interpreters(Robert Nystrom):インタプリタの作り方.Web 版は無料.https://craftinginterpreters.com

22.2 型システム

- 『型システム入門 —— プログラミング言語と型の理論』(Benjamin C. Pierce,原題 Types and Programming Languages,通称 TAPL):型システムの定番教科書.
- elaboration-zoo(András Kovács,GitHub):依存型の型検査器を小さく作る例集.双方向型検査の実装の参考になる.

22.3 定理証明

- Theorem Proving in Lean 4,Functional Programming in Lean:Lean 公式の無料教材.
- Software Foundations:Rocq(Coq)で証明を学ぶ定番教材.

22.4 論文(発展)

- Edwin Brady, Idris 2: Quantitative Type Theory in Practice (2021):数量型理論.
- Sebastian Ullrich, Leonardo de Moura, Counting Immutable Beans (2019):Lean 4 の参照カウント.
- Alex Reinking ほか, Perceus: Garbage Free Reference Counting with Reuse (2021):Koka の参照カウント.

23 用語集

用語	意味
AST	抽象構文木.プログラムの構造を木の形で表したもの
s 式	括弧で囲んだ前置記法の式.Lisp の書き方
束縛	値に名前を付けること
イミュータブル	一度作った値を変更できないこと
クロージャ	関数と,それが使う外側の変数をまとめたもの
型推論	書かれていない型をコンパイラが推測すること
双方向型検査	検査モードと推論モードを組み合わせる型検査の方式
オーバーフロー	整数が型の範囲を超えること
篩型	条件を満たす値だけを持つ型
依存型	型の中に値が入る型
カリー=ハワード同型対応	命題を型,証明をプログラムとみなす対応
タクティク	証明を組み立てる命令
停止性	関数が必ず終わること
消去	コンパイル後に証明などを取り除くこと
数量型理論	値を何回使うかを型に書く仕組み(0・1・ω)
線形型	ちょうど 1 回使う値の型
アフィン型	たかだか 1 回使う値の型.Rust の所有権
参照カウント	参照されている数を数えてメモリを片付ける方式
GC	使われていないメモリを自動で探して片付ける仕組み
未定義動作	言語仕様が結果を保証しない動作
メモリ安全	未定義動作が起きないこと
末尾呼び出し	関数の最後の処理としての関数呼び出し
LLVM IR	LLVM が受け取る中間表現
クロージャ変換	クロージャを関数ポインタと環境のペアに変換すること
SMT ソルバー	論理式が成り立つかを自動で判定するプログラム(Z3 など)

24 コンパイラを作るときに考えること

最後に,手書きでコンパイラを作るときに考えておくべきこと,気をつけることをまとめる.

24.1 進め方

24.1.1 1 つの機能を最後まで通す

「字句解析を全部作る → 構文解析を全部作る → …」と段階ごとに完成させていくと,最後の段階に着くまで何も動かない.途中で仕様が変わると,それまでの作業がやり直しになる.

代わりに,**小さな機能 1 つを最初から最後まで通す**進め方がおすすめである.

1. 整数と `(+ 1 2)` だけを,字句解析 → 構文解析 → 実行まで通す.
2. `let` を足して,同じく最後まで通す.
3. 関数を足す.`if` を足す.…

毎回「動くもの」があるので,間違いにすぐ気づけるし,やる気も続きやすい.

24.1.2 仕様を固めすぎない

grammer.md は何度も書き換わってきた.実装してみて初めて分かる問題(書きにくい,構文解析しにくい,など)は多い.仕様は「実装しながら直すもの」と考え,実装より先に細部まで決めすぎないほうがよい.

24.1.3 こまめにコミットする

うまく動いている状態で git にコミットしておけば,変更で壊れても戻れる.「FizzBuzz が動いた」のような区切りでコミットするとよい.

24.2 文法の設計

24.2.1 文法を形式的に書く

grammer.md は例で文法を説明しているが,構文解析器を書くには「何が書けて,何が書けないか」をはっきりさせる必要がある.**EBNF** という記法で書いておくと,構文解析器をそのまま書き起こせる.

```
expr     = literal | ident | sexpr ;
sexpr    = "(" ( let | if | match | loop | apply ) ")" ;
let      = "let" [ "mut" ] typed_name "<-" expr ;
typed_name = "[" type ident [ "|" expr ] "]" ;
apply    = expr { expr } ;
```

24.2.2 曖昧さを残さない

同じ書き方が 2 通りに読めると,構文解析器が困るし,読む人も困る.topazus で今気になるところは次のとおり.

- `|` が篩型の条件と関数本体の両方に使われている.
- `(< x -1)` の `<` と `-` が `<-` と読まれる.
- `[array i64 n a]` の長さ `n` と名前 `a` の区切り.

新しい構文を追加するときは,「既存の構文と混ざらないか」を毎回確認する.

24.2.3 予約語を早めに決める

`let`,`if`,`match` などの予約語は,あとから増やすと,その名前を変数に使っていたコードが壊れる.予約語の一覧は早めに決めておく.

24.3 エラー処理

24.3.1 位置情報を持たせる

今の tokeniser.rs の `Token` は,ソースコードの何行目何文字目かを覚えていない.これでは「3 行目の 18 文字目が間違っている」というエラーが出せない.皮肉なエラー文も,場所が分からないとただの悪口になってしまう.

トークンと AST のすべてに**位置情報(Span)**を持たせるのがよい.あとから追加すると全部を書き直すことになるので,早めにやっておく.

```
#[derive(Debug, Clone, Copy)]
pub struct Span { pub line: usize, pub col: usize }

pub struct Spanned<T> { pub node: T, pub span: Span }
```

24.3.2 panic しない

`unwrap()` や `panic!` を使うと,間違ったコードを入力したときにコンパイラ自体が落ちる.ユーザーの間違いは `Result` で返して,分かりやすいエラー文にする.`unwrap()` は「絶対に起きない」と分かっている場所だけに使う.

24.3.3 最初のエラーで止まらない

エラーが 1 つ見つかるたびに止まると,10 個の間違いを直すのに 10 回コンパイルすることになる.構文エラーの場合は「次の `)` まで読み飛ばして続ける」などの方法で,なるべく多くのエラーを一度に報告できるとよい.ただしこれは後回しでもよい.

24.4 Rust で書くときの注意

24.4.1 段階ごとにデータ構造を分ける

AST(構文解析の結果)に型情報や名前解決の結果を後から書き込もうとすると,`Option` だらけになったり,借用チェッカーとの戦いになったりする.

段階ごとに**別のデータ構造**を作り,「AST を受け取って新しいデータを返す」形にすると整理しやすい.

```
ソース → Vec<Token> → AST → 名前解決済みAST → 型付きIR → C/LLVM
```

24.4.2 名前は文字列のまま扱わない

変数名を `String` のまま比較・コピーしていると遅く,同じ名前の別の変数(シャドーイング)も区別できない.名前解決の段階で,変数ごとに番号(ID)を振るとよい.

24.4.3 構文解析器は位置で管理する

`Peekable<Iter<Token>>` は 1 つ先しか見られない.2 つ以上先を見たい場面(`[i64 n | ...]` の `|` があるかどうかなど)が出てくるので,`Vec<Token>` と現在位置 `pos: usize` を持つ構造体にしておくと扱いやすい.

```
struct Parser { tokens: Vec<Token>, pos: usize }
impl Parser {
```

```
    fn peek(&self, n: usize) -> Option<&Token> { self.tokens.get(self.pos + n) }
}
```

24.5 テスト

24.5.1 サンプルコードと期待する出力を並べる

`tests/` に「topazus のコード」と「期待する出力」のペアを置き,全部まとめて実行して比較するテストを作る.言語に機能を足したときに,前に動いていたものが壊れていないかをすぐ確認できる.

```
tests/fizzbuzz.to        // topazus のコード
tests/fizzbuzz.expected  // 期待する出力
```

間違ったコードが**正しくエラーになるか**のテストも同じくらい大事である.特に型検査や篩型では,「通るべきでないコードが通ってしまう」バグが一番怖い.

24.5.2 インタプリタとコード生成の結果を比べる

インタプリタを先に作っておくと,C 出力の正しさを「インタプリタと同じ結果になるか」で確認できる.これを**差分テスト**という.

24.6 型検査と証明の信頼性

24.6.1 型検査器のバグは証明を壊す

topazus では,型検査器が「証明が正しいか」を判定する.型検査器にバグがあると,間違った証明が通ってしまい,「証明済み」という保証がすべて意味を失う.実行時チェックを消している部分では,そのまま範囲外アクセスなどにつながる.

定理証明系の言語では,証明を最終的に検査する部分(**カーネル**)をできるだけ小さく保つ.小さければ読んで確かめられるからである.Lean,Rocq もこの設計になっている.topazus でも,タクティクなどの便利な機能はカーネルの外に置き,カーネルは単純に保つとよい.

24.6.2 変数の取り違え

依存型の型検査では,式の中の変数を別の式で置き換える処理(**代入**)を何度も行う.このとき,名前がたまたま同じ別の変数を取り違えるバグが非常に起きやすい.対策として,変数を名前ではなく「何番目の外側で束縛されたか」という番号で表す **de Bruijn インデックス**がよく使われる.elaboration-zoo で詳しく扱っている.

24.6.3 逃げ道を作りすぎない

`unsafe` や `partial` のような逃げ道は便利だが,増やすほど保証が弱くなる.逃げ道を足すときは「これで何が保証されなくなるか」を毎回 grammer.md に書いておく.

24.7 コード生成

24.7.1 出力する C に未定義動作を入れない

C に変換する場合,**出力した C のコード自体**が未定義動作を含まないように注意する.

- C の符号付き整数のオーバーフローは未定義動作である.折り返し演算子 `+%` は,符号なし整数で計算してから戻すなどして実装する.
- オーバーフロー検査には `__builtin_add_overflow` などの gcc/clang の組み込み関数が使える.
- 範囲チェックを省いてよいのは,証明がある場所だけにする.

24.7.2 数学用の型が実行時に残る場合

`[nat]` や `[int]` は上限がないので,証明の中だけでなく実行時に使われる場合は多倍長整数の実装が必要になる.「数学用の型は証明の中でしか使えない」と制限するか,多倍長整数を用意するかを決めておく.

24.8 競プロで使うための注意

24.8.1 入出力の速さ

競プロでは 10^5 行以上の入出力がよくある.`println` のたびに画面に書き出していると,それだけで時間切れになることがある.出力は**バッファ**にためて,最後にまとめて書き出す実装にする.入力も同様に,まとめて読み込んでから区切る.

24.8.2 計測する

「速いはず」ではなく,実際に時間を測る.同じ問題を C++と topazus で解いて比べると,どこが遅いかが分かる.参照カウントの増減やコピーが予想外に多い,というのはよくある原因である.

24.8.3 標準ライブラリ

競プロでは,言語そのものより標準ライブラリ(ソート,優先度付きキュー,ハッシュマップ,二分探索など)の充実度が使いやすさを左右する.言語の機能が揃ってきたら,よく使うデータ構造から用意していく.

24.9 まとめ:よくある落とし穴

落とし穴	対策
全段階を作るまで何も動かない	小さな機能を最後まで通す
エラーの場所が分からない	最初から Span を持たせる
コンパイラ自体が panic で落ちる	`Result` でエラーを返す
機能追加で前のものが壊れる	サンプルと期待出力のテスト
間違ったコードが通る	エラーになるべきコードのテスト
型検査器が大きくなり信頼できない	カーネルを小さく保つ
依存型で変数を取り違える	de Bruijn インデックス
出力した C に未定義動作	符号なし演算・組み込み関数を使う
競プロで入出力が遅い	バッファリング
仕様を決めすぎて進まない	実装しながら仕様を直す

言語作りは時間がかかる.全部を完成させることより,作りながら「なぜこうなっているのか」が分かっていくこと自体を楽しんでほしい.